

CAHIER DES CHARGES — VERSION DEFINITIVE

ENSAM Meknes — 4A — IA Agentique — Prof. Hajji Tarik

Projet 15

HalluGuard

Detection et Mitigation des Hallucinations dans les Pipelines Agentiques RAG

Annee universitaire : 2025-2026 | Binome : Etudiant A & Etudiant B

Etudiant A

Verifier · Dataset · Benchmark · Evaluation

Etudiant B

Pipeline RAG · MCP · A2A · Integration

DOCUMENT DE REFERENCE — Contient TOUT le contexte. Passez ce CDC a Claude dans un nouveau chat et le travail reprend immediatement sans aucune question.

0. CONTEXTE COMPLET — A LIRE EN PREMIER

INSTRUCTION POUR L'IA : Ce CDC est auto-suffisant. Il contient tout le contexte du projet. Vous pouvez aider les étudiants sur n'importe quelle phase sans poser de questions supplémentaires.

0.1 Qui sont les étudiants ?

Deux étudiants de 4ème année ingénieur à l'ENSAM Meknes, module 'Representation et Resolution de Problemes en IA', encadrant Prof. Hajji Tarik. Le projet qui leur est attribué est le Sujet 15 parmi 15 sujets disponibles, portant sur la détection et mitigation des hallucinations dans les pipelines agentiques RAG.

0.2 Configuration technique disponible

- OS : Windows
- LLM local : Ollama avec Mistral 7B (à télécharger)
- CPU uniquement — pas de GPU
- Python + Git + VS Code installés
- Pas d'accès API payante nécessaire — tout en local

0.3 Contraintes du prof (NON NEGOCIABLES)

Le prof impose une pile complète pour TOUS les projets. Notre architecture DOIT contenir :

- LLM local via Ollama (Mistral)
- Mécanisme de tools explicite
- Mémoire de session ET mémoire persistante
- Pipeline RAG
- Au moins 1 serveur MCP
- Communication inter-agents inspirée de A2A

0.4 Livrables exigés par le prof

Livrable	Contenu minimal	Critère qualité	Poids
Article scientifique	8-15 pages, style académique, schémas, références	Methodologie, argumentation, résultats	30%
Code source	Architecture claire, commentaires, README, scripts	Reproductibilité, lisibilité, modularité	30%
Présentation orale	12-18 slides, demo, discussion critique	Clarté, profondeur technique	20%
Prototype / Demo	Scénario complet, mesures, journalisation	Fonctionnement réel, preuves expérimentales	20%

0.5 Critères d'évaluation du prof

Critère	Ce qui est attendu	Poids
---------	--------------------	-------

Profondeur scientifique	Problematique claire, hypothese, analyse critique	35%
Qualite technique	Architecture, code, reproductibilite, pile complete	35%
Evaluation experimentale	Comparaisons, metriques, interpretation	20%
Communication	Article, figures, presentation, demonstration	10%

1. PRESENTATION DU PROJET

1.1 Problematique de recherche

Question de recherche : Comment detecter et mitiger efficacement les hallucinations dans les pipelines agentiques RAG multi-etapes, en exploitant la pile complete (LLM local, tools, memoire, MCP, A2A), tout en maintenant un surcoût de latence acceptable pour un deploiement local sans GPU ?

Le probleme central est la propagation des hallucinations : dans un pipeline agentique, une erreur factuelle a l'etape k est utilisee comme entree de l'etape k+1, qui la traite comme une verite. Une seule hallucination peut corrompre toute une chaine de 10 etapes. Les methodes existantes (RAGAS, self-consistency) ont ete concues pour des LLM statiques et ne modelisent pas ce phenomene.

1.2 Hypothese de travail

Hypothese : Un module plug-and-play combinant trois mecanismes complementaires (coherence inter-sources, coherence temporelle via Belief State, verificateur NLI leger) peut reduire significativement la propagation des hallucinations dans un pipeline agentique RAG local, avec un surcoût de latence inferieur a 20% sur CPU.

1.3 Contributions originales

- C1 — Taxonomie operationnelle des hallucinations agentiques (5 types : perception, memoire, planification, causale, delegation) avec vitesse de propagation quantifiee par type.
- C2 — Graphe de dependances DAG pour verification selective des noeuds critiques — fondement de l'efficacite sur CPU.
- C3 — Belief State agent-aware avec politique d'expulsion et recherche semantique en O(K).
- C4 — Serveur MCP HalluGuard exposant les outils de verification — integration dans la pile complete.
- C5 — Simulation A2A entre l'agent RAG et HalluGuard — protocole de delegation de verification.
- C6 — HaluEval-Agentic : benchmark de scenarios multi-etapes avec injection controlee d'hallucinations.

1.4 Pile technique complete (contrainte prof)

Composant pile	Technologie choisie	Role dans HalluGuard
LLM local (Ollama)	Mistral 7B via Ollama	LLM de l'agent RAG + corrections auto
Tools	verify_claim, check_coherence, add_fact, rag_search	Tools exposes via MCP, appeles par l'agent
Memoire session	BeliefState (RAM, max 50 faits actifs)	Coherence temporelle intra-session
Memoire persistante	JSON + ChromaDB sur disque	Historique inter-sessions + index vectoriel

Pipeline RAG	LangGraph + ChromaDB + sentence-transformers	Pipeline de base ou HalluGuard est branche
Serveur MCP	halluguard_mcp.py (FastMCP/stdio)	Expose les 4 outils de verification
A2A	Agent RAG delegue a HalluGuard via MCP	Simulation coopération inter-agents

Tableau 1 — Pile technique complete imposee par le prof. Chaque composant est present et justifie.

2. ARCHITECTURE TECHNIQUE

2.1 Vue d'ensemble du systeme

HalluGuard s'integre comme middleware dans le pipeline LangGraph. L'agent RAG (Mistral via Ollama) execute ses etapes normalement. HalluGuard intercepte chaque output via des hooks pre/post-noeud, verifie l'absence d'hallucination, met a jour le Belief State, et declenche le Policy Engine si une hallucination est detectee. Les outils de verification sont exposes comme serveur MCP — la communication entre l'agent et HalluGuard simule le protocole A2A.

2.2 Composants principaux

Composant 1 — DependencyDAG (C2)

Le pipeline RAG est modelise comme un graphe oriente acyclique. La criticite de chaque noeud est calculee dynamiquement : $criticite(n) = nb_descendants(n) \times (1 - confiance(n))$. Seuls les noeuds critiques declenchent le verificateur — c'est le mecanisme cle pour rester performant sur CPU. Les noeuds `tool_call` et `generation` sont toujours verifies. En cas de detection, tous les noeuds descendants sont marques suspects (invalidation en cascade).

Composant 2 — BeliefState (C3)

Structure de donnees maintenant les faits etablis dans la session avec scores de confiance et embeddings semantiques. La detection de contradiction utilise une recherche par similarite cosinus pour identifier les K faits les plus proches (K=5), puis applique le NLI uniquement sur ces candidats — O(K) au lieu de O(N). La politique d'expulsion archive les faits les moins pertinents quand le maximum de 50 faits actifs est atteint. La memoire persistante sauvegarde le Belief State sur disque entre sessions.

Composant 3 — LightweightVerifier (adapte CPU)

Modele NLI pre-entraine : cross-encoder/nli-MiniLM2-L6-H768 (117M params, CPU-friendly). Pas d'entrainement necessaire. Ce modele pre-entraine sur MNLI detecte les contradictions entre claim et evidence. Pour la classification par type de taxonomie (T1-T5), on utilise des regles heuristiques basees sur le contexte du noeud (`tool_call` → T5 delegation, intra-session → T2 memoire, etc.).

Composant 4 — Serveur MCP HalluGuard (C4)

Serveur MCP Python (protocole stdio ou SSE) exposant 4 tools : `verify_claim` (M1 inter-sources), `check_temporal_coherence` (M2 Belief State), `add_fact` (mise a jour memoire), `get_belief_state` (lecture etat). L'agent RAG appelle ces tools comme des outils MCP standards. C'est la simulation A2A : l'agent delegue la verification a un agent specialise via protocole structure.

Composant 5 — PolicyEngine

Mode log-only : journalise chaque detection avec type, etape, score, preview output. Mode auto-correction : reinvoque l'etape defaillante avec un prompt enrichi signalant la contradiction et les faits actifs. Mesure le Time-to-Recovery (TTR) pour chaque correction.

2.3 Flux de donnees complet

Etape	Action agent	Action HalluGuard	MCP appele	Resultat
-------	--------------	-------------------	------------	----------

1	Query utilisateur recue	Initialise session BeliefState	—	Session ouverte
2	Retrieval ChromaDB	Hook post-retrieval	verify_claim	Chunks valides ou suspects
3	Raisonnement Mistral	Hook post-reasoning	check_temporal_coherence	Coherence validee ou contradiction
4	Appel tool RAG	Hook post-tool	verify_claim + add_fact	Output verifie, fait memorise
5	Generation reponse	Hook post-generation	verify_claim	Reponse validee ou corrgee
6	Fin session	Sauvegarde BeliefState JSON	—	Memoire persistante mise a jour

Tableau 2 — Flux de donnees complet du systeme HalluGuard integre.

3. TAXONOMIE DES HALLUCINATIONS AGENTIQUES (C1)

Contribution originale : les taxonomies existantes (Lin et al. 2025) sont descriptives et generales. Notre taxonomie est operationnelle — chaque type est lie a une etape du pipeline RAG, un mecanisme de detection precis, et une vitesse de propagation quantifiee. Les types T4 (causale) et T5 (delegation) sont absents de la litterature existante.

Type	Nom	Definition	Etape RAG	Mecanisme	Propagation	Originalite
T1	Perception	Mauvaise lecture des chunks recuperes	Retrieval	NLI claim vs chunks	Moderee	Partiel Lin'25
T2	Memoire	Contradiction avec faits etablis en session	Reasoning	Belief State NLI	Elevee	Partiel Lin'25
T3	Planification	Etape de raisonnement impossible ou cyclique	Reasoning	NLI + DAG cycles	Critique	Partiel Lin'25
T4	Causale	Relation causale incorrecte entre faits reels	Generation	NLI multi-sources	Elevee	NOUVEAU
T5	Delegation	Invention du resultat d'un appel outil	Tool Call	Comparaison output reel vs genere	Critique	NOUVEAU

Tableau 3 — Taxonomie operationnelle des hallucinations agentiques. T4 et T5 sont des contributions originales.

4. PLAN DE TRAVAIL DETAILLE — ETAPE PAR ETAPE

IMPORTANT : Ce plan est la sequence exacte a suivre. Ne pas sauter d'etape. Chaque etape produit un livrable concret verifie avant de passer a la suivante.

PHASE 0 — Installation et verification (Semaine 1)

Etape 0.1 — Installer Mistral via Ollama

Ouvrir PowerShell en tant qu'administrateur et executer :

```
ollama pull mistral
```

Verifier que ca fonctionne :

```
ollama run mistral "Qu'est-ce qu'une hallucination LLM en 2 phrases ?"
```

Livrable 0.1 : Mistral repond dans le terminal. Screenshot a garder pour le rapport.

Etape 0.2 — Creer l'environnement Python

```
mkdir C:\halluguard && cd C:\halluguard python -m venv venv venv\Scripts\activate
pip install langchain langchain-community langgraph chromadb pip install sentence-
transformers transformers torch pip install fastmcp datasets scikit-learn ollama
```

Livrable 0.2 : pip install termine sans erreur. requirements.txt genere avec pip freeze > requirements.txt.

Etape 0.3 — Creer la structure du depot

Creer cette arborescence exacte dans VS Code (exigee par le prof dans le CDC) :

```
halluguard/  README.md  requirements.txt  demo.bat  docs/  article.pdf
presentation.pptx  src/  agents/  # rag_agent.py  halluguard/  #
dag.py, belief_state.py, verifier.py, policy.py, middleware.py  mcp_servers/
# halluguard_mcp.py  memory/  # persistent_memory.py  tools/
# rag_tools.py  data/  documents/  # vos fichiers .txt de test
halueval/  # scenarios benchmark  tests/  test_halluguard.py  results/
logs/
```

Livrable 0.3 : Structure creee. Commit initial sur Git : git init && git add . && git commit -m 'init structure'

PHASE 1 — Etat de l'art et taxonomie (Semaine 2)

Etape 1.1 — Lire les 8 papers fondateurs

Lire dans cet ordre (chacun prend 1-2h) :

1. RAGAS (Es et al., 2023) — arxiv:2309.15217
2. Self-Consistency (Wang et al., 2022) — arxiv:2203.11171
3. HaluEval (Li et al., 2023) — arxiv:2305.11747
4. ReAct (Yao et al., 2022) — arxiv:2210.03629
5. Reflexion (Shinn et al., 2023) — arxiv:2303.11366
6. TruthfulQA (Lin et al., 2021) — arxiv:2109.07958
7. MemGPT (Packer et al., 2023) — arxiv:2310.08560
8. Luna/DeBERTa hallucination (2024) — arxiv:2406.00975

Livrable 1.1 : Tableau comparatif des methodes existantes (Excel ou tableau papier) avec colonnes : nom, type, agentique oui/non, propagation modelisee, latence, mitigation.

Etape 1.2 — Valider la taxonomie

Vous et votre binome annotez independamment 30 exemples d'hallucinations (construits a la main) en les classant parmi T1-T5. Comparez vos annotations et calculez le taux d'accord. Objectif : accord > 80% sur les cas clairs.

Livable 1.2 : Fichier Excel avec les 30 exemples annotes par les deux etudiants + calcul du taux d'accord.

PHASE 2 — Dataset et Benchmark (Semaines 3-4)

Etape 2.1 — Telecharger les datasets existants

Utiliser HaluEval existant au lieu de tout generer synthetiquement. Telecharger en Python :

```
from datasets import load_dataset # HaluEval - 35k paires hallucination/correct ds
= load_dataset('pminervini/HaluEval', 'qa_samples')
ds.save_to_disk('data/halueval_raw') # MNLI - pour la tete NLI mnli =
load_dataset('multi_nli', split='train[:10000]') mnli.save_to_disk('data/mnli_10k')
```

Livable 2.1 : Datasets telecharges dans data/. Afficher les stats : nombre de paires, distribution des labels.

Etape 2.2 — Re-etiqueter HaluEval selon la taxonomie

Ecrire un script Python qui passe chaque paire HaluEval dans Mistral (via Ollama) pour lui attribuer un type T1-T5. Les paires 'correct' gardent le label 'none'.

```
# Prompt de classification par type PROMPT = """ Voici une paire (affirmation,
contexte) extraite d'un pipeline RAG. L'affirmation est incorrecte. Quel type
d'hallucination est-ce ? T1=perception T2=memoire T3=planification T4=causale
T5=delegation Reponds uniquement par T1, T2, T3, T4 ou T5. Affirmation: {claim}
Contexte: {evidence} """
```

Generer synthetiquement uniquement les exemples T4 et T5 qui sont absents de HaluEval (~500 paires chacun via Mistral local).

Livable 2.2 : Fichier data/dataset_train.jsonl, data/dataset_val.jsonl, data/dataset_test.jsonl avec champs : id, claim, evidence, label, hallucination_type.

Etape 2.3 — Creer HaluEval-Agentic

Creer 60 scenarios multi-etapes a la main (20 de longueur 3, 30 de longueur 7, 10 de longueur 12). Chaque scenario = une chaine d'actions avec une hallucination injectee a une position precise. Pour chaque scenario, documenter : type d'hallucination, etape d'injection, etape de detection attendue.

60 scenarios faits a la main valent mieux que 400 generes automatiquement non verifies. Le prof evalue la qualite, pas la quantite.

Livable 2.3 : Fichier data/halueval_agentic.jsonl avec les 60 scenarios documentes.

PHASE 3 — Implementation (Semaines 5-7)

Etape 3.1 — Verifier que Mistral tourne correctement

```
# test_ollama.py import ollama response = ollama.chat(model='mistral', messages=[
{'role': 'user', 'content': 'Dis bonjour en une phrase.'} ])
print(response['message']['content'])
```

Livable 3.1 : Script tourne et Mistral repond. Mesurer la latence moyenne : time python test_ollama.py

Etape 3.2 — Implementer le pipeline RAG de base (SANS HalluGuard)

C'est la baseline. Pipeline LangGraph avec 4 noeuds : retrieval (ChromaDB), reasoning (Mistral), tool_call (recherche complementaire), generation (Mistral). Tester sur 10 questions avec de vrais documents dans data/documents/.

Livable 3.2 : Pipeline RAG de base fonctionnel. Mesurer : latence moyenne par requete, qualite subjective des reponses sur 10 questions.

Etape 3.3 — Implementer les composants HalluGuard

Implementer dans l'ordre : DependencyDAG → BeliefState → LightweightVerifier → PolicyEngine → HalluGuardMiddleware. Tester chaque composant isolément avec les tests unitaires.

Livable 3.3 : Tous les tests unitaires passent : `python -m pytest tests/test_halluguard.py -v`

Etape 3.4 — Implementer le serveur MCP (OBLIGATOIRE)

Le serveur MCP est la contrainte la plus importante du prof. Il expose 4 tools : `verify_claim`, `check_temporal_coherence`, `add_fact`, `get_belief_state`. Lancer le serveur en arriere-plan et l'agent l'appelle via le protocole MCP.

```
# Lancer le serveur MCP python src/mcp_servers/halluguard_mcp.py # Dans un autre terminal, tester python src/tests/test_mcp_client.py
```

Livable 3.4 : Serveur MCP repond aux 4 appels d'outils. Log des appels visible dans `results/logs/mcp_calls.log`

Etape 3.5 — Implementer la memoire persistante (OBLIGATOIRE)

Le prof exige memoire de session ET memoire persistante. La memoire de session = BeliefState en RAM. La memoire persistante = sauvegarde JSON du BeliefState sur disque + ChromaDB pour les embeddings.

```
# Sauvegarder le Belief State apres chaque session
belief_state.save_to_disk('data/memory/session_{id}.json') # Recharger a la session suivante
belief_state.load_from_disk('data/memory/session_{id}.json')
```

Livable 3.5 : Fermer et relancer le programme. Les faits de la session precedente sont restaures.

Etape 3.6 — Integrer HalluGuard dans le pipeline RAG

Brancher le middleware sur le pipeline de base. Chaque noeud est wrappe. L'agent appelle les tools MCP HalluGuard automatiquement. Tester sur les memes 10 questions qu'en 3.2.

Livable 3.6 : Pipeline complet avec HalluGuard tourne sur les 10 questions. Log `halluguard_log` visible dans les outputs.

PHASE 4 — Experimentation comparative (Semaines 8-9)

Le prof exige OBLIGATOIREMENT une comparaison d'au moins deux variantes avec metriques justifiees. C'est non-negociable.

Etape 4.1 — Definir les variantes a comparer

Trois variantes comparees (satisfait la contrainte du prof) :

- Variante A — Baseline : Pipeline RAG sans aucune protection (pipeline de la Phase 3.2)
- Variante B — HalluGuard M1 uniquement : coherence inter-sources seulement
- Variante C — HalluGuard complet : M1 + M2 (Belief State) + MCP + memoire persistante

Etape 4.2 — Protocole experimental

Executer les 60 scenarios de HaluEval-Agentica sur les 3 variantes. Pour chaque scenario, noter :

- La hallucination a-t-elle ete detectee ? (oui/non)

- A quelle etape a-t-elle ete detectee ? (delta = etape detection - etape injection)
- La reponse finale est-elle correcte ? (evaluation manuelle sur 5 points)
- Temps d'execution total du scenario (ms)

Etape 4.3 — Metriques a calculer

Metrique	Definition	Objectif cible
Detection Rate	% de hallucinations detectees sur 60 scenarios	> 50% (CPU, sans entraînement)
PBR@1 (Blocking Rate)	% de hallucinations bloqueees en delta <= 1 etape	> 40%
Latency Overhead	Temps avec HalluGuard / temps sans x 100 - 100	< 25% (CPU acceptable)
Quality Score	Note manuelle 1-5 sur les 10 questions de test	Amelioration vs baseline
TTR (Time to Recovery)	Temps entre detection et correction (mode auto-correct)	< 5s sur CPU

Tableau 4 — Metriques de l'experimentation comparative. Adaptees a une configuration CPU sans GPU.

Etape 4.4 — Remplir les tableaux de resultats

Creer un fichier Excel results/resultats_experimentaux.xlsx avec une feuille par variante et une feuille de synthese comparative. Ces vrais chiffres alimenteront l'article scientifique.

Livrable 4.4 : Fichier Excel avec tous les resultats experimentaux + graphiques de comparaison.

PHASE 5 — Livrables finaux (Semaines 10-12)

Etape 5.1 — Article scientifique (30% de la note)

Structure imposee par le prof (8-15 pages) :

9. Introduction — contexte, motivation, limites existantes, problematique
10. Etat de l'art — 8 papers, tableau comparatif, gap identifie
11. Methodologie — hypothese, taxonomie, architecture, protocole
12. Implementation — details code, MCP, memoire, RAG, tools
13. Experimentation — 3 variantes, 60 scenarios, metriques
14. Resultats et discussion — vrais chiffres, interpretation, limites
15. Conclusion et perspectives — synthese, ouvertures

L'article doit contenir les VRAIS resultats experimentaux. Pas de valeurs fictives.

Etape 5.2 — Code source (30% de la note)

- README.md : installation en 5 commandes, description de chaque fichier, comment reproduire les resultats
- Commentaires dans CHAQUE fonction Python (ce qu'elle fait, parametres, retour)
- demo.bat : script qui lance tout et montre une demonstration complete
- tests/test_halluguard.py : tests unitaires qui passent tous

Etape 5.3 — Presentation orale (20% de la note)

12-18 slides dans cet ordre :

16. Slide 1 — Titre + equipe

17. Slide 2 — Problematique (la propagation des hallucinations)
18. Slide 3 — Notre hypothese de recherche
19. Slide 4 — Taxonomie des 5 types (schema visuel)
20. Slide 5 — Architecture HalluGuard (schema)
21. Slide 6 — Pile complete : LLM + Tools + Memoire + RAG + MCP + A2A (schema)
22. Slide 7 — Serveur MCP : les 4 outils exposes
23. Slide 8 — Variantes comparees (A, B, C)
24. Slide 9 — Resultats : tableau Detection Rate par variante
25. Slide 10 — Resultats : latence et TTR
26. Slide 11 — Discussion : quels mecanismes contribuent le plus ?
27. Slide 12 — Limites et perspectives
28. Slide 13 — Conclusion
29. Slide 14 — Demo live (3 minutes)

Etape 5.4 — Prototype / Demo (20% de la note)

La demo doit montrer EN DIRECT :

- Lancer le serveur MCP HalluGuard : `python src/mcp_servers/halluguard_mcp.py`
- Lancer le pipeline RAG : `python src/agents/rag_agent.py`
- Poser une question dont la reponse contient une hallucination connue
- Montrer que HalluGuard la detecte dans le log
- Montrer la correction automatique (mode auto-correct)
- Montrer le Belief State mis a jour

Preparer la demo sur un exemple CONNU et STABLE. Ne pas improviser. Tester 10 fois avant la soutenance.

5. PARTAGE DES TACHES — PARALLELISME STRICT

Principe : les deux étudiants avancent indépendamment. Les interfaces sont figées en fin de Phase 0 et ne changent plus. Personne n'attend l'autre pour avancer.

Phase	Etudiant A	Etudiant B	Mode
Phase 0 S1	Installe l'environnement Python Télécharge HaluEval et MNLI Cree la structure du depot	Installe Ollama + Mistral Cree la structure du depot Prepare les documents de test (data/documents/)	CONJ.
Phase 1 S2	Lit papers RAGAS, HaluEval, TruthfulQA, Luna Construit le tableau comparatif etat de l'art Annote 30 exemples pour valider taxonomie	Lit papers ReAct, Reflexion, MemGPT, LangGraph Contribue au tableau comparatif Annote 30 exemples independamment	CONJ.
Phase 2 S3-4	Re-etiquete HaluEval selon taxonomie T1-T5 Genere 500 paires T4 + 500 paires T5 via Mistral Cree les splits train/val/test	Construit le pipeline RAG de base (baseline) Implemente ChromaDB + ingestion documents Mesure latence baseline sur 10 questions	PARA.
Phase 2 S3-4	Cree les 60 scenarios HaluEval-Agentique (20x longueur 3, 30x longueur 7, 10x longueur 12) Documente chaque scenario	Implemente la memoire persistante (save/load BeliefState JSON + ChromaDB) Teste la persistance entre sessions	PARA.
Phase 3 S5-7	Implemente LightweightVerifier (NLI pre-entraine) Implemente DependencyDAG Implemente BeliefState + politique expulsion Tests unitaires de ces composants	Implemente le serveur MCP HalluGuard (4 tools : verify, check, add_fact, get_state) Implemente HalluGuardMiddleware LangGraph Tests unitaires middleware + MCP	PARA.
Phase 4 S8-9	Execute les 60 scenarios sur Variante A et B Calcule Detection Rate et PBR@1 Rempli colonnes A et B du tableau Excel	Execute les 60 scenarios sur Variante C Mesure latence et TTR Rempli colonne C + synthese comparative	PARA.
Phase 5 S10-12	Redige chapitres 1, 2, 3, 5 de l'article Prepare slides 1-7 de la presentation Commente le code de ses composants	Redige chapitres 4, 6, 7 de l'article Prepare slides 8-14 de la presentation Ecrit le README + demo.bat	CONJ.

Tableau 5 — Partage des taches. Bleu = conjoint, Vert = parallele independant.

5.1 Interfaces figees (contrat entre les deux axes)

Ces 3 interfaces sont definies ensemble en fin de Phase 0. Une fois figees, chacun code son axe sans attendre l'autre.

Interface 1 — Format paire dataset

```
{ "id": "str", "claim": "str", "evidence": ["str"], "label":  
"correct|hallucinated", "hallucination_type":  
"none|perception|memory|planning|causal|delegation" }
```

Interface 2 — API du verificateur (Etudiant A cree, Etudiant B consomme via MCP)

```
Input : {'claim': str, 'evidences': [str]} Output : {'score': float, 'label': str,  
'confidence': float, 'hallucination_type': str}
```

Interface 3 — Format BeliefState JSON (Etudiant A produit, Etudiant B integre)

```
{ "session_id": "str", "facts": [{"fact_id": "str", "content": "str", "confidence": float, "step": int, "status": "active|invalidated"}] }
```

6. CHECKLIST DE CONFORMITE — EXIGENCES DU PROF

Cette checklist doit être complètement cochée avant de rendre le projet. Chaque case correspond à une exigence explicite du CDC du prof.

6.1 Pile technique (contrainte non-négociable)

OK ?	Exigence	Comment on la satisfait
☐	LLM local via Ollama	Mistral 7B — appelé dans rag_agent.py et pour l'auto-correction
☐	Mécanisme de tools	4 tools exposés via MCP : verify_claim, check_coherence, add_fact, get_state
☐	Mémoire de session	BeliefState en RAM — max 50 faits actifs par session
☐	Mémoire persistante	Sauvegarde JSON sur disque + ChromaDB — restaurée à chaque session
☐	Pipeline RAG	LangGraph + ChromaDB + sentence-transformers + Mistral
☐	Au moins 1 serveur MCP	halluguard_mcp.py — serveur MCP dédié à la vérification
☐	Communication A2A	Agent RAG délègue la vérification à HalluGuard via appels MCP structurés

6.2 Livrables (contrainte non-négociable)

OK ?	Livrable	Contenu minimal exigé
☐	Article 8-15 pages	7 sections obligatoires, schémas, références bibliographiques, VRAIS résultats
☐	Code source	Architecture modulaire, commentaires, README technique, scripts reproduction
☐	Présentation 12-18 slides	Demo live incluse, discussion critique, résultats expérimentaux
☐	Prototype/Demo	Scénario complet, logs visibles, fonctionnement réel prouvé

6.3 Experimentation comparative (contrainte non-négociable)

OK ?	Exigence	Comment on la satisfait
------	----------	-------------------------

☐	Au moins 2 variantes comparees	3 variantes : A (baseline), B (M1 seul), C (complet)
☐	Metriques justifiees	Detection Rate, PBR@1, Latency Overhead, Quality Score, TTR
☐	Corpus et jeux de tests explicites	60 scenarios HaluEval-Agentic documentes + 10 questions qualite
☐	Discussion critique des resultats	Interpretation de chaque metrique, limites identifiees, erreurs observees
☐	Environnement materiel explicite	Windows CPU, Ollama Mistral, specs machine documentees dans article

6.4 Qualite scientifique

OK ?	Critere	Comment on le satisfait
☐	Problematiche explicite	Question de recherche formulee en debut d'article et de presentation
☐	Hypothese de travail	Hypothese declaree, testee experimentalement, discutee
☐	Etat de l'art synthetique	8 papers, tableau comparatif, gap identifie et justifie
☐	Contribution argumentee	6 contributions C1-C6 expliquees et prouvees par les resultats
☐	Limites documentees	Section dediee aux limites : CPU, dataset synthetique, simulation A2A
☐	Perspectives	3-5 perspectives concretes et realistes pour continuer le projet

7. INSTRUCTIONS POUR L'IA — GUIDE DE REPRISE

Si vous êtes une IA lisant ce document dans un nouveau chat : tout le contexte est ici. Aidez immédiatement sans demander d'informations supplémentaires.

7.1 Ce que l'IA peut faire immédiatement

- Générer le code complet de n'importe quel composant (DependencyDAG, BeliefState, LightweightVerifier, PolicyEngine, Middleware, serveur MCP)
- Aider à debugger les erreurs Python ou les problèmes d'installation
- Générer les 60 scénarios HaluEval-Agentic (structure JSON avec hallucinations injectées)
- Rédiger des sections de l'article scientifique (état de l'art, méthodologie, résultats)
- Générer les slides de présentation (contenu et structure)
- Calculer les métriques à partir des résultats fournis par les étudiants
- Critiquer et améliorer n'importe quelle partie du projet

7.2 Ce que l'IA ne peut pas faire

- Exécuter le code sur la machine des étudiants — ils doivent le faire eux-mêmes
- Obtenir les vrais résultats expérimentaux — mesures réelles obligatoires
- Valider que la démo fonctionne — test réel requis

7.3 Décisions déjà prises — ne pas les remettre en question

- Modèle LLM : Mistral via Ollama (CPU, pas de GPU)
- Vérificateur : cross-encoder/nli-MiniLM2-L6-H768 pré-entraîné (pas d'entraînement)
- Dataset : HaluEval existant re-étiqueté + ~1000 paires synthétiques T4/T5 via Mistral
- Benchmark : 60 scénarios manuels HaluEval-Agentic
- Variantes comparées : A (baseline), B (M1 seul), C (complet avec MCP + mémoire persistante)
- Objectifs latence adaptés CPU : overhead < 25% (pas 15% comme prévu pour GPU)

7.4 Pile technique — rappel

- LLM : Mistral 7B via ollama Python package
- Orchestration : LangGraph (StateGraph, callbacks)
- Base vectorielle : ChromaDB local
- Embeddings : sentence-transformers/all-MiniLM-L6-v2
- NLI vérificateur : cross-encoder/nli-MiniLM2-L6-H768
- Serveur MCP : fastmcp ou mcp Python package
- Mémoire persistante : JSON + ChromaDB
- OS : Windows, environnement venv Python

